Lab 9: data pipelines in Microsoft Fabric that run without you

AI Engineer (Data) course · about 90 minutes · everything in the browser

What you will build

One pipeline that keeps FreshCart's support tickets up to date. Each run copies only the new tickets from the Lab 7 database into your lakehouse (files and tables in one place). It checks each ticket and parks the bad ones with a reason, then gives the good ones a category from a model. The steps run in order, so once you switch on a schedule nobody has to press Run. At the end you ask the Lab 9 tables questions in plain English.

> **support_tickets** (Azure SQL) → **tickets_raw** → **tickets_clean** + **tickets_quarantine** → **tickets_enriched** → answers from a data agent

Part (time)	Goal: what you have at the end
Step 0. A lakehouse (5 min)	freshcart_lake: the one place every table of today lands.
A. Incremental loading: copy only what is new (15 min)	tickets_raw with 200,000 tickets, then 600 more when the trainer adds a day: the copy job copies only the new ones.
B. Data quality: check, keep, park (25 min)	Four rules check every ticket, set up by clicking in a Dataflow (Power Query in Fabric): 200,578 clean, 22 parked in tickets_quarantine, each with its reason.
C. LLM enrichment: a model label (15 min)	Every clean ticket has an AI category, for 10 model calls instead of 200,578, because each distinct text is labelled once.
D. Orchestration: put it in order (10 min)	A pipeline, ingest_tickets, that runs the checks, then the labels only if the checks succeeded, and tries a failed step again.
E. Data agents: ask your lakehouse (20 min)	Your own Fabric data agent (an assistant that turns a question into a query) answers questions about a copy of the Lab 9 tables in the class workspace, and you check its answers.

> Fill in the results table at the end as you go, and hand it in. Labs 1 to 4 in Azure, your documents, is the other Day 9 handbook.

Before you start

- **The data.** FreshCart's 200,000 support tickets live in the Lab 7 database, freshcart_lab7, table **support_tickets**. New tickets arrive every day.
- **Your workspace.** Your trainer gives you a Fabric workspace of your own: **FreshCart Lab 9 - your name**. Open **app.fabric.microsoft.com** and sign in with your class account.
- **No keys anywhere.** Fabric reads the database and the lakehouse as you, and the notebook calls the class model as you.
- **The Lab 9 files** are in the **lab9_fabric** folder: the Part C notebook to import, the optional Part B notebook, and data_agent_instructions.txt for Part E. Part E's optional code is in lab8_foundry.

Step 0. A lakehouse (5 minutes)

Why this part. Every step today writes a table, and the copy job, the dataflow, the notebook and the data agent all need to read each other's tables. A **lakehouse** is that one shared place: files plus tables, stored as Delta tables, which keep a history of every change.

Make the lakehouse

1. Open **app.fabric.microsoft.com**. In the left bar click **Workspaces** and pick **FreshCart Lab 9 - your name**, not My workspace.
2. Click **New item**, type **Lakehouse** in the search box, and pick **Lakehouse**.
3. Name it **freshcart_lake**. Leave the schema option off, then click **Create**.

> No Lakehouse in New item, only Report, Dashboard and Semantic model? You are in a workspace with no Fabric capacity, such as My workspace. Open Workspace settings, License info: your class workspace says Trial.

Part A. Incremental loading: copy only what is new (15 minutes)

Why this part. The tickets live in the Lab 7 database, the system the support team uses every day. We work on our own copy, so our checks never slow it down. Copying all 200,000 tickets every day would be slow and wasteful, so the copy job copies only the tickets that are new since last time. You will also see the one kind of change this misses.

Set up the copy job

1. In your workspace click **New item**, then **Copy job**.
2. Source: pick **Azure SQL database**.
3. Server: **freshcart-mini-sql.database.windows.net**.
4. Database: **freshcart_lab7**.
5. Authentication: **Organizational account** (you are already signed in).
6. Click **Next**. The first connection can take a minute while the database wakes up.
7. Tick **dbo.support_tickets**.
8. Destination: pick your lakehouse, **freshcart_lake**.
9. Settings: pick **Incremental copy**, and incremental column **created_at**.
10. Write method: **Merge**, key column **ticket_id**.
11. Map to destination: name the table **tickets_raw**.
12. Click **Save**, then **Run**.

Read run 1

1. Open **Results**. Write down the rows read, 200,000 in our run, and the **Upper bound**: the newest created_at it copied.

Runs 2 and 3

1. Click **Run** again. This time it should read 0 rows: nothing is new.
2. Wait while your trainer adds a new day of tickets to the database.
3. Click **Run** a third time. It should read 600 rows: only the new day.

Look at the edited ticket

1. Open **freshcart_lake**, then switch to **SQL analytics endpoint** at the top right.
2. Click **New SQL query** and paste this:

```
SELECT refund_amount FROM tickets_raw WHERE ticket_id = 'TKT-0000042'
```

3. Click **Run**. It should say 103.17. Your trainer changed this ticket in the database to 999.00, and the copy did not pick that up.

What we saw (just read)	Why it matters
Run 1: 200,000 rows, 80 s	A full load. The copy job remembers the newest created_at, 2024-12-30 23:59: the watermark.
Run 2: 0 rows, 63 s	Nothing new, nothing copied. Running twice is safe.
Run 3: 600 rows	Only the new day, from the old watermark to the new one.
TKT-0000042: source 999.00, lakehouse 103.17	The trainer also changed one OLD ticket. A copy keyed on created_at never looks at it again. Write down how you would catch edits.

Part B. Data quality: check, keep, park, with a Dataflow (25 minutes)

Why this part. Some tickets arrive broken: no customer, a negative refund, a category nobody knows. If they reach a report or the model, the numbers are wrong and nobody can tell why. Deleting them would hide the problem, so instead each bad ticket is parked in its own table with the reason, where someone can fix it. You learn to write each rule once and check every ticket the same way on every run.

A **Dataflow Gen2** is Fabric's point-and-click tool for cleaning data, built on Power Query. You build the four checks as columns, then send the good tickets to one table and the bad ones, with the rule they broke, to another.

Connect to tickets_raw

1. In your workspace click **New item**, then **Dataflow Gen2**. Name it **quality_tickets** and click **Create**.
2. Click **Get data from another source**, search **Lakehouse** and pick it.
3. Connection: **Create new connection**, authentication **Organizational account** (you are already signed in). Click **Next**.
4. Open your workspace, then **freshcart_lake**. Tick **tickets_raw** and click **Create**.

Make the query that checks

1. In the **Queries** list on the left, right-click **tickets_raw** and choose **Reference**. A new query appears.
2. In **Query settings** on the right, change its Name to **tickets_checked**.
3. Right-click the **ticket_id** column header and choose **Remove duplicates**: a ticket that arrived twice is checked once.

Add the four rule columns

1. Click **Add column**, then **Conditional column**.
2. Type the new column name from the table below.
3. On the If line, pick the Column name and Operator, then type the Value. Leave the small pencil menus as they are, except in rule_order.
4. Type the Output, then the Else. Where the table says null, type the word **null**: it means empty.
5. For rule_category only, click **Add clause** until there are 8 lines, one per known category.
6. Click **OK**. The new column appears at the right end of the table, showing null in most rows.
7. Repeat steps 1 to 6 for each row of the table, until you have four rule columns.

New column name	If ... then ..., Else
rule_customer	customer_id **equals** null, then **no customer**; Else null
rule_refund	refund_amount **is less than** 0, then **negative refund**; Else null
rule_order	resolution_minutes **is less than** first_response_minutes (in the Value box, open the pencil menu and pick **Use values in a column**), then **resolved before first response**; Else null
rule_category	category **equals** missing_item, then null; one more clause each for late_delivery, payment, spillage, other, account, food_safety and wrong_item, each then null; Else **unknown category**

Each Output names the problem the ticket has: a ticket with no customer gets **no customer**. A ticket that passes shows null in that column.

One difference from the notebook: it parks a ticket whose refund or minutes are empty, and these rules let it through. The class data has no such empty values, so both give the same 22.

Join the rules into one column

1. Click the **rule_customer** header, then Shift-click **rule_category**, so all four rule columns are selected.
2. Click the **Transform** tab at the top. The data type button has no label: it shows **ABC 123 Any**. Open it and pick **Text**.
3. Click **Add column**, then **Custom column**. Name it **failed_rules**, set **Data type** to **Text**, and paste this one line, which joins the four into one and skips the empty ones:

```
Text.Combine({[rule_customer], [rule_refund], [rule_order], [rule_category]}, "; ")
```

4. Click **OK**. A clean ticket has an empty failed_rules; a bad one lists the rules it broke.
5. Click the **rule_customer** header again, Shift-click **rule_category**, right-click one of them, and choose **Remove columns**.

Split into clean and parked

1. In the **Queries** list, right-click **tickets_checked** and choose **Reference**. In Query settings, rename the new query **tickets_clean**.
2. Open the **failed_rules** filter arrow, tick only **(blank)**, and click **OK**. These are the tickets that broke no rule.
3. Right-click the **failed_rules** header and choose **Remove columns**.

4. Right-click **tickets_checked** again, choose **Reference**, and rename the new query **tickets_quarantine**.
5. Open its **failed_rules** filter, untick **(blank)**, and click **OK**. The status bar at the bottom should say **Rows: 22**.
6. Leave **tickets_checked** as it is: both new queries read from it.

Send both to the lakehouse

1. Select **tickets_clean** in the Queries list.
2. Under **Data destination** at the bottom right, click **+**, then **Lakehouse**, then **Next**.
3. Open your workspace and pick **freshcart_lake** (or its **dbo** folder, if it shows one).
4. Type the table name **tickets_clean** and click **Next**.
5. Keep **Use automatic settings** and click **Save settings**.
6. Select **tickets_quarantine** and repeat steps 2 to 4, with table name **tickets_quarantine**.
7. Before you save, look at **Column mapping**: failed_rules must be ticked and say **Text**. If it says Any and is unticked, click Cancel, set the failed_rules type to Text (Transform, the data type button), repeat step 6, then check again.
8. Keep **Use automatic settings** and click **Save settings**. Give no destination to tickets_raw or tickets_checked.

Save and run

1. Click **Home**, then **Save and run**. The dataflow runs for about 2 minutes.

Check it finished

1. Go back to your workspace and hover over the **quality_tickets** row.
2. Click the **...** that appears next to its name, then **Recent runs**.
3. A panel lists each run with its start time, how long it took and its status. Wait until the status says **Succeeded**. A green tick and a Refreshed time on the row mean the same.

Look at the result

1. Open **freshcart_lake** from the workspace list: the Lakehouse row, not the SQL analytics endpoint row.
2. On the left, under **Tables**, find **tickets_clean** and **tickets_quarantine** (inside **dbo**, if your lakehouse shows schemas).
3. Click **tickets_quarantine**. It should have 22 rows, each with a failed_rules value such as no customer.
4. Click **tickets_clean**. It has no failed_rules column. The preview shows only the first 1,000 rows.
5. To count, switch to **SQL analytics endpoint** at the top right.
6. Click **New SQL query** and paste this:

```
SELECT COUNT(*) FROM tickets_clean
```

7. Click **Run**. It should say 200,578.

Run it again

1. In your workspace, open the **...** on quality_tickets and click **Refresh now**.
2. When Recent runs says Succeeded, run the count again and open tickets_quarantine. Both totals should be the same as run 1.

What we saw (just read)	Why it matters
Run 1: 200,578 clean, 22 parked, 97 s	The 22 parked tickets are the trainer's planted problems: 9 with no customer, 4 with a negative refund, 6 with an unknown category, 3 resolved before the first response, each named in failed_rules.
Run 2: totals unchanged, 67 s	With automatic settings each run replaces both tables, so a second run never stores a ticket twice. The price: it checks all 200,600 again every time, instead of only the new ones. Fine at this size; for a large table, Dataflows offer incremental refresh.

Optional, the same in code: the notebook **lab9_1_quality.ipynb** in the lab9_fabric folder does Part B in Python and checks only the tickets it has not seen. Use it or the dataflow, not both: they write the same two tables.

Part C. LLM enrichment: a model label for every clean ticket (15 minutes)

Why this part. Support wants complaints counted by type, but the category staff typed in may be wrong. A model can read each ticket's text and give it a category. Asking the model about each of 200,578 tickets, every day, would be slow and cost money each time, so the notebook asks once per distinct text and saves the answer for next time. You learn to keep model cost under control, and you will see what the labels reveal about the recorded categories.

Open the notebook

1. In your workspace click **Import**, then **Notebook**, then **From this computer**.
2. Pick **lab9_2_enrich.ipynb** from the lab9_fabric folder and click **Upload**. It appears in your workspace list.
3. Open it. In **Explorer** on the left, click **Add data items** and pick **freshcart_lake**.
4. In the **Settings** cell, paste the address your trainer gives you into **FOUNDRY**.

Run it twice

1. Click **Run all** and wait until the last cell finishes.
2. Read the cost cell. Write down the model calls and the cost.
3. Click **Run all** again, and read the cost cell again.

What we saw (just read)	Why it matters
Run 1: 10 model calls, $0.000248	The 200,578 clean tickets have only 10 distinct texts. We label each text once and keep the answer, keyed on model, prompt version and text.
Run 2: 0 calls, $0.0	Everything came from the cache: cheap, and the same label every time.
One call per ticket: $4.97	And 200,578 calls. Estimate on a sample and multiply before you run the whole table.
Recorded category agrees with the label: 12.4%	About 1 in 8, what chance would give. The categories in this data were never reliable; the label shows it.

Part D. Orchestration: put it in order, without you (10 minutes)

Why this part. So far you ran each step by hand. With new tickets every day, someone would have to remember to run the checks and then the labels, in that order. If the labels ran first, bad tickets would be labelled too. A pipeline runs the steps in order, starts a step only if the one before it succeeded, and tries a failed step again. You learn what makes a pipeline safe to leave running every night.

Build the pipeline

1. In your workspace click **New item**, then **Pipeline**, and name it **ingest_tickets**.
2. On the toolbar click **Dataflow**. In its **Settings**, pick your workspace and **quality_tickets**.
3. On the toolbar click **Notebook**. In its **Settings**, pick **lab9_2_enrich**.
4. Drag from the green tick on the dataflow box to the notebook box. The arrow means the notebook runs only if the dataflow succeeded.
5. Click each box in turn. On its **General** tab, set **Retry** to 1 and the retry interval to 60 seconds.

Optional: run the copy job first

1. On the pipeline toolbar, open **Activities** and pick **Copy job**.
2. In its **Settings**, under **Connection**, click **Browse all**, then pick **Copy job**.
3. Choose **Organizational account** and sign in. You do this once; the connection is kept.
4. Pick your workspace and your copy job from Part A (named **CopyJob_1** unless you renamed it).
5. Drag from the green tick on the copy job box to the dataflow box.

   The pipeline now runs the same copy job, with the same watermark, so the whole day runs in order: copy, check, label. Without this step you run the copy job yourself, as in Part A. Not yet tested in class.

Run it

1. Click **Validate**, then **Run**.
2. Look at **Schedule** on the pipeline and on the copy job. Leave both off.

A pipeline with only the dataflow step ran in 128 seconds in our test; the notebook step adds its own run time and, with nothing new, makes 0 model calls.

Part E. Data agents: ask your lakehouse with a Fabric data agent (20 minutes)

Why this part. The tables now hold the answers a manager wants, such as how many tickets were parked and why. A manager will not write SQL, so a data agent writes and runs it for them from a plain-English question. Before anyone trusts it, you test it with questions whose answers you already know.

Fabric's **Data agent** answers questions about your tables in plain English: you pick the tables and write the instructions, and it writes and runs the SQL. It does not run on the class trial capacity (the screen says an admin must change the SKU type), so Part E runs in the class workspace **FreshCart data agent demo (trainer)**, on a small paid capacity your trainer switches on for class. Its lakehouse holds a copy of the same Lab 9 tables, so pick the freshcart_lake in this workspace, not your own. You each make one agent, named after your account. Everyone can see and change every agent here: open only your own, and never edit freshcart_lake.

Open the class workspace

1. In Fabric click **Workspaces** and pick **FreshCart data agent demo (trainer)**. If you cannot see it, your trainer has not added your account yet.

Make your data agent

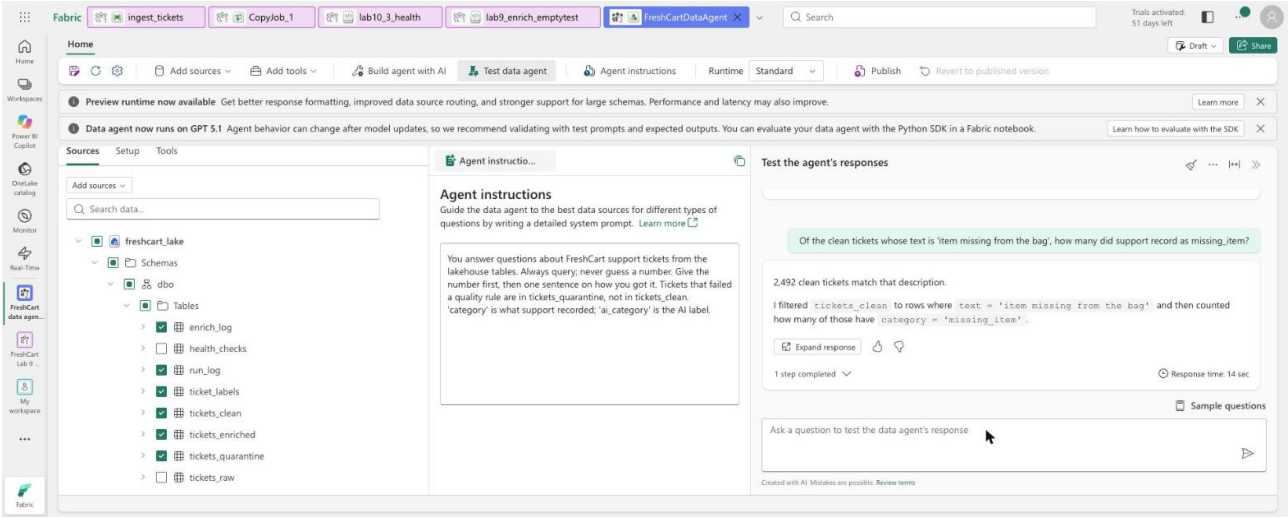
1. Click **New item**, then **Data agent**. Name it after your account, such as **itc-ai-1-data-agent**, and click **Create**.
2. Click **Add a data source**. Two freshcart_lake rows may show: one is your own.
3. Click **>>** next to Explorer in that window, and pick **FreshCart data agent demo (trainer)**.
4. Pick its **freshcart_lake** and click **Add**. If Explorer still says No sources added after a few seconds, reload the page.
5. In Explorer open freshcart_lake, then **Schemas**, **dbo**, **Tables**, and tick the six tables: **enrich_log, run_log, ticket_labels, tickets_clean, tickets_enriched, tickets_quarantine**.
6. Reload the page once and open Explorer again: the six boxes must still be ticked. If they are empty, tick them again. With no table ticked, every answer says An error occurred.
7. Click into **Agent instructions**, select all the example text, and replace it with these instructions. They are also in **data_agent_instructions.txt** in the lab9_fabric folder (the same ones the optional code below uses). The draft saves itself; do not use **Save as**, which makes a copy.

You answer questions about FreshCart support tickets. Always query; never guess a number. Give the number first, then one sentence on how you got it. Tickets that failed a quality rule are in tickets_quarantine, not in tickets_clean. tickets_quarantine has one row per parked ticket, so count its rows for the total; failed_rules names each problem the ticket has, separated by '; '. To count one reason, count the rows whose failed_rules contains its name; a ticket with two problems counts once under each, so those counts can add up to more than the total. 'category' is what support recorded; 'ai_category' is the AI label.

Test it

1. In **Test the agent's responses**, ask the five questions below, one at a time.
2. Under each answer, open **1 step completed** to see the SQL it ran. Write down the answer and whether it is right.
3. Ask question 1 again, and compare the two answers.

Question	Right answer
1. How many tickets were put in quarantine, and for which reasons?	22: 9, 4, 6 and 3
2. Which ticket text is the most common, and how many clean tickets have it?	refund has not reached my account, 20,225
3. Of the clean tickets whose text is 'item missing from the bag', how many did support record as missing_item?	2,492
4. How many clean tickets were created in December 2024?	17,152
5. In the first AI labelling run, how many model calls were made and what did it cost?	10 calls, $0.000248



A data agent: the tables it may read on the left, the instructions in the middle, question 3 answered on the right.

Optional, the same in code: lab9_3_ask_the_lake.py in your lab8_foundry folder asks the same five questions of your own Lab 9 lakehouse, 3 times each, and scores them. Add **LAKE_SQL_ENDPOINT=** your lakehouse's SQL address (on the freshcart_lake SQL analytics endpoint, **...**, **Copy SQL connection string**) to lab8_foundry's .env, then run **python lab9_3_ask_the_lake.py**, and again with **--fixed**, which adds one sentence describing enrich_log. Ours: 37 of 45 as given, 45 of 45 with --fixed. Run it before Lab 10 Part C, which adds two days of tickets.

Fabric data agent, or your own agent in Foundry?

- **Use the Fabric data agent** when people only need answers from data that is already in Fabric. It takes minutes, needs no code, writes the SQL itself and runs it with the asker's own permissions. It needs an F2 or larger capacity, and it can be published to Copilot.
- **Build your own agent in Foundry** when you need more control: your choice of model, tools beyond Fabric (a refund policy, an email, another system), your own checks such as read-only, and tests that run before every change, as in Lab 10. It takes code. The agent itself needs no Fabric capacity, but any Fabric data it reads still does.
- **You can also do both.** A Foundry agent can call a published data agent as one of its tools. The class has not tested this yet.

Your results table (hand in)

	Run 1	Run 2	After the new day
Copy job: rows read			
Quality: clean / parked			
Labels: model calls and cost			
Data agent: right of 5; question 1 asked again, same answer?			

Add one line: how you would catch an edit to an old ticket, like TKT-0000042.

What the class takes away

A pipeline you can trust runs twice without harm: the copy keeps a watermark, the checks rebuild the same two tables however often they run, and the model work is cached by content. Bad rows are parked with a reason, not dropped. Every run writes a log line. And a watermark only sees new rows: edits and deletions need their own plan.

If something goes wrong

What you see	What to do
The copy job waits a long time at Connecting	The database pauses after an hour; the first call wakes it. Wait a minute and try again.
Table or view not found: tickets_raw	The notebook has no lakehouse attached: Explorer, Add data items, freshcart_lake.
HTTP 401 or 403 from the model	Check the FOUNDRY address; your class account needs access to the class Foundry resource. Tell the trainer.
The SQL endpoint does not show a new table	It catches up within a minute or two. Refresh it.
The pipeline fails on a notebook	Open the failed activity's output: the notebook's own error is there.
The dataflow says a table already exists	In Data destination choose **Existing table**, pick it, and keep automatic settings: it is replaced on each run.
tickets_quarantine has no failed_rules column	Its type was Any, so the destination left it out. Set the type to Text and check Column mapping, as in Send both to the lakehouse, then run again.
Every data agent answer says An error occurred	No table is ticked. Open Explorer: the source should be the demo workspace's freshcart_lake, with its six tables ticked.
tickets_quarantine shows 0 rows, or all of them	Check the Else of each rule is null, not empty, and that the filter on failed_rules keeps or drops (blank) as the step says.